Mass No. 1

Create a webpage using Tailwind CSS with a heading and paragraph. Apply text color, font size, and spacing utilities. (5M)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Tailwind Example</title>
  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
  <div class="bg-white p-8 rounded-2xl shadow-lg max-w-lg text-center">
    <!-- Heading -->
    <h1 class="text-3xl font-bold text-blue-600 mb-4">
      Welcome to Tailwind CSS
    </h1>
    <!-- Paragraph -->
    <p class="text-gray-700 text-lg leading-relaxed">
      This is a simple webpage created using Tailwind CSS. It demonstrates
      how to apply text color, font size, and spacing utilities easily.
    </p>
  </div>
</body>
</html>
```

Develop a Laravel form with Name and Email fields. Apply validation (required, email) and display error messages. (10M)

Add this in routes/web.php:

```
use App\Http\Controllers\FormController;

Route::get('/form', [FormController::class, 'showForm']);
Route::post('/form', [FormController::class, 'submitForm'])->name('form.submit');
```

Create controller:

```
php artisan make:controller FormController
```

code:-> Controller

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class FormController extends Controller
{
    public function showForm()
    {
        return view('form');
    }
    public function submitForm(Request $request)
    {
        $request->validate([
            'name' => 'required',
            'email' => 'required|email',
        ]);
        return back()->with('success', 'Form submitted successfully!');
    }
}
```

resources/views/form.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Laravel Form</title>
</head>
<body>
    <h2>Contact Form</h2>
    <!-- Success Message -->
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('form.submit') }}" method="POST">
        @csrf
        <!-- Name -->
        <div>
            <label>Name:</label><br>
            <input type="text" name="name" value="{{ old('name') }}">
            @error('name')
```

```
            <p style="color:red;">{{ $message }}</p>
        @enderror
    </div>
    <br>
    <!-- Email -->
    <div>
        <label>Email:</label><br>
        <input type="text" name="email" value="{{ old('email') }}">
        @error('email')
            <p style="color:red;">{{ $message }}</p>
        @enderror
    </div>
    <br>
    <button type="submit">Submit</button>
  </form>
</body></html>
```

Create a Model and Controller for Student table. Fetch and display all records in a Blade view. (10M)

Create Model + Migration

Run this command:

```
php artisan make:model Student -m
```

database/migrations/...create_students_table.php:

```
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->timestamps();
    });
}
```

Run migration:

```
php artisan migrate
```

app/Models/Student.php)

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model

{

    protected $fillable = ['name', 'email'];
```

```
}
```

Create Controller

```
php artisan make:controller StudentController

namespace App\Http\Controllers;

use App\Models\Student;

class StudentController extends Controller

{

    public function index()

    {

        $students = Student::all();

        return view('students.index', compact('students'));

    }

}
```

Route (web.php)

```
use App\Http\Controllers\StudentController;
Route::get('/students', [StudentController::class, 'index']);
```

resources/views/students/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Students List</title>
</head>
<body>
    <h2>All Students</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
        </tr>
        @foreach($students as $student)
        <tr>
            <td>{{ $student->id }}</td>
            <td>{{ $student->name }}</td>
            <td>{{ $student->email }}</td>
        </tr>
        @endforeach
    </table>
</body>
```

Mass No. 2

Design a button using Tailwind CSS with background color, padding, rounded corners, and hover effect. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tailwind Button</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="flex items-center justify-center h-screen bg-gray-100">
    <button class="bg-blue-500 text-white px-6 py-2 rounded-lg hover:bg-blue-700 transition duration-
300">
        Click Me
    </button>
</body>
</html>
```

Create a Laravel form to upload an image. Store it in storage and display it in a Blade view. (10M)

Create Route (web.php)

```
use App\Http\Controllers\ImageController;

Route::get('/upload', [ImageController::class, 'index']);
Route::post('/upload', [ImageController::class, 'store'])->name('image.upload');
```

Create Controller

```
php artisan make:controller ImageController
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class ImageController extends Controller
{
    public function index()
    {
        return view('upload');
    }
    public function store(Request $request)
    {
```

```
        $request->validate([
            'image' => 'required|image|mimes:jpg,jpeg,png|max:2048'
        ]);
        $path = $request->file('image')->store('images', 'public');
        return back()->with('success', 'Image uploaded successfully!')->with('image', $path);
    }
}
```

Create Storage Link

Run this command (important):

```
php artisan storage:link
```

(resources/views/upload.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Upload Image</title>
</head>
<body>
    <h2>Upload Image</h2>
    <!-- Success Message -->
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <!-- Upload Form -->
    <form action="{{ route('image.upload') }}" method="POST" enctype="multipart/form-data">
        @csrf
        <input type="file" name="image">
        @error('image')
            <p style="color:red;">{{ $message }}</p>
        @enderror

        <br><br>
        <button type="submit">Upload</button>
    </form>
    <!-- Display Image -->
    @if(session('image'))
        <h3>Uploaded Image:</h3>
        <img src="{{ asset('storage/' . session('image')) }}" width="300">
```

```
    @endif
</body>
</html>
```

Create a Model and Controller for Product table and insert a new record using Eloquent ORM. (10M)

Create Model + Migration

```
php artisan make:model Product -m
```

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->decimal('price', 8, 2);
        $table->timestamps();
    });
}
```

Run migration:

```
php artisan migrate
```

(app/Models/Product.php)

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Product extends Model
{
 protected $fillable = ['name', 'price'];
}
```

Create Controller

```
php artisan make:controller ProductController
```

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Product;
class ProductController extends Controller
{
    public function create()
    {
        return view('products.create');
    }
    public function store(Request $request)
    {
```

```
        $request->validate([
            'name' => 'required',
            'price' => 'required|numeric'
        ]);
        Product::create([
            'name' => $request->name,
            'price' => $request->price
        ]);
        return back()->with('success', 'Product added successfully!');
    }
}
```

(web.php)

```
use App\Http\Controllers\ProductController;
Route::get('/products/create', [ProductController::class, 'create']);
Route::post('/products/store', [ProductController::class, 'store'])->name('products.store');
```

resources/views/products/create.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Product</title>
</head>
<body>
    <h2>Add Product</h2>
    <!-- Success Message -->
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('products.store') }}" method="POST">
        @csrf
        <!-- Name -->
        <div>
            <label>Product Name:</label><br>
            <input type="text" name="name" value="{{ old('name') }}">
            @error('name')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
```

```
        <br>
  <!-- Price -->
        <div>
            <label>Price:</label><br>
            <input type="text" name="price" value="{{ old('price') }}">
            @error('price')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
        <br>
        <button type="submit">Add Product</button>
    </form>
</body>
</html>
```

Mass No. 3

Create a card component using Tailwind CSS with title, description, border, and shadow. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tailwind Card</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <div class="max-w-sm bg-white border border-gray-200 rounded-xl shadow-lg p-6">
        <!-- Title -->
        <h2 class="text-2xl font-bold text-gray-800 mb-2">
            Card Title
        </h2>
        <!-- Description -->
        <p class="text-gray-600 text-base leading-relaxed">
            This is a simple card component created using Tailwind CSS.
            It includes a title, description, border, and shadow for a clean UI.
```

```
    </p>
  </div></body></html>
```

Create a migration for employees table with fields: name, email, and salary. Run the migration. (10M)

Create Migration

Run this command:

```
php artisan make:migration create_employees_table
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
  public function up()
  {
    Schema::create('employees', function (Blueprint $table) {
      $table->id();
      $table->string('name');
      $table->string('email')->unique();
      $table->decimal('salary', 10, 2);
      $table->timestamps();
    });
  }
  public function down()
  {
    Schema::dropIfExists('employees');
  }
};
```

Run Migration

Execute:

```
php artisan migrate
```

Create a Controller to fetch all records from Employee table and display them. (10M)

Create Model

```
php artisan make:model Employee
```

(app/Models/Employee.php)

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Employee extends Model
{
   protected $fillable = ['name', 'email', 'salary'];
}
```

Create Controller

```
php artisan make:controller EmployeeController
```

```
namespace App\Http\Controllers;

use App\Models\Employee;

class EmployeeController extends Controller

{

   public function index()

   {

      $employees = Employee::all(); // Fetch all records

      return view('employees.index', compact('employees'));

   }

}
```

web.php)

```
use App\Http\Controllers\EmployeeController;

Route::get('/employees', [EmployeeController::class, 'index']);
```

(resources/views/employees/index.blade.php)

```
<!DOCTYPE html>

<html>

<head>

   <title>Employee List</title>

</head>

<body>

   <h2>All Employees</h2>

   <table border="1" cellpadding="10">

      <tr>

         <th>ID</th>

         <th>Name</th>

         <th>Email</th>

         <th>Salary</th>

      </tr>

      @foreach($employees as $emp)
```

```
        <tr>
            <td>{{ $emp->id }}</td>
            <td>{{ $emp->name }}</td>
            <td>{{ $emp->email }}</td>
            <td>{{ $emp->salary }}</td>
        </tr>
        @endforeach
    </table>
</body>
</html>
```

Mass No. 4

Design a login form UI using Tailwind CSS and center it on the screen. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <div class="bg-white p-8 rounded-2xl shadow-lg w-full max-w-sm">
        <!-- Heading -->
        <h2 class="text-2xl font-bold text-center text-gray-800 mb-6">
            Login
        </h2>
        <!-- Form -->
        <form>
            <!-- Email -->
            <div class="mb-4">
                <label class="block text-gray-700 mb-1">Email</label>
                <input type="email" placeholder="Enter your email"
                    class="w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
blue-400">
```

```
        </div>
        <!-- Password -->
        <div class="mb-4">
            <label class="block text-gray-700 mb-1">Password</label>
            <input type="password" placeholder="Enter your password"
                class="w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
blue-400">
        </div>
        <!-- Button -->
        <button type="submit"
            class="w-full bg-blue-500 text-white py-2 rounded-lg hover:bg-blue-600 transition
duration-300">
            Login
        </button>
    </form>   </div> </body></html>
```

Create a Laravel form with validation rules (required, min length). Show validation errors. (10M)

(web.php)

```
use App\Http\Controllers\UserController;

Route::get('/user-form', [UserController::class, 'showForm']);
Route::post('/user-form', [UserController::class, 'submitForm'])->name('user.submit');
```

Create Controller

```
php artisan make:controller UserController
```

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UserController extends Controller
{
    public function showForm()
    {
        return view('user-form');
    }
    public function submitForm(Request $request)
    {
        $request->validate([
            'name' => 'required|min:3',
            'password' => 'required|min:6'
        ]);
```

```
        return back()->with('success', 'Form submitted successfully!');
    }
}
```

resources/views/user-form.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>User Form</title>
</head>
<body>
    <h2>User Form</h2>
    <!-- Success Message -->
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('user.submit') }}" method="POST">
        @csrf
        <!-- Name -->
        <div>
            <label>Name:</label><br>
            <input type="text" name="name" value="{{ old('name') }}">
            @error('name')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
        <br>
        <!-- Password -->
        <div>
            <label>Password:</label><br>
            <input type="password" name="password">
            @error('password')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
        <br>
        <button type="submit">Submit</button>
    </form>
```

```
</body>
</html>
```

Create a Controller to update a record in User table using Eloquent ORM. (10M)

```
php artisan make:controller UserController
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\User;
class UserController extends Controller
{
    public function edit($id)
    {
        $user = User::findOrFail($id);
        return view('users.edit', compact('user'));
    }
    public function update(Request $request, $id)
    {
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email'
        ]);

        $user = User::findOrFail($id);
        $user->update([
            'name' => $request->name,
            'email' => $request->email
        ]);
        return redirect('/users')->with('success', 'User updated successfully!');
    }      }
```

(web.php)

```
use App\Http\Controllers\UserController;

Route::get('/users/edit/{id}', [UserController::class, 'edit']);
Route::post('/users/update/{id}', [UserController::class, 'update'])->name('users.update');
```

(resources/views/users/edit.blade.php)

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Edit User</title>
</head>
<body>
    <h2>Edit User</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('users.update', $user->id) }}" method="POST">
        @csrf
        <!-- Name -->
        <div>
            <label>Name:</label><br>
            <input type="text" name="name" value="{{ old('name', $user->name) }}">
            @error('name')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
        <br>
        <!-- Email -->
        <div>
            <label>Email:</label><br>
            <input type="text" name="email" value="{{ old('email', $user->email) }}">
            @error('email')
                <p style="color:red;">{{ $message }}</p>
            @enderror
        </div>
        <br>
        <button type="submit">Update</button>
    </form>
</body>
</html>
```

Mass No. 5

Create three boxes in a row using Tailwind flex utilities with spacing. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flex Boxes</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <!-- Flex Container -->
    <div class="flex space-x-6">
        <!-- Box 1 -->
        <div class="bg-blue-500 text-white p-6 rounded-lg shadow-md">
            Box 1
        </div>

        <!-- Box 2 -->
        <div class="bg-green-500 text-white p-6 rounded-lg shadow-md">
            Box 2
        </div>
        <!-- Box 3 -->
        <div class="bg-red-500 text-white p-6 rounded-lg shadow-md">
            Box 3
        </div>
    </div>
</body>
</html>
```

Create a migration to add a new column (phone) to customers table. (10M)

Create Migration

Run this command:

```
php artisan make:migration add_phone_to_customers_table --table=customers
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Support\Facades\Schema;

return new class extends Migration {

    public function up()

    {

        Schema::table('customers', function (Blueprint $table) {

            $table->string('phone')->nullable()->after('email');

        });

    }

    public function down()

    {

        Schema::table('customers', function (Blueprint $table) {

            $table->dropColumn('phone');

        });

    }

};
```

Run Migration

```
php artisan migrate
```

Create a Resource Controller for Order (id, customer_id, product_name, quantity, price) and define routes (Index, Create, Store). (10M)

Create Model + Migration

```
php artisan make:model Order -m
```

Edit migration file:

```
public function up()
{
    Schema::create('orders', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('customer_id');
        $table->string('product_name');
        $table->integer('quantity');
        $table->decimal('price', 10, 2);
        $table->timestamps();
    });
}
```

php artisan migrate

(app/Models/Order.php)

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Order extends Model

{

    protected $fillable = [
```

```
        'customer_id',
        'product_name',
        'quantity',
        'price'
    ];
}
```

Create Resource Controller

```
php artisan make:controller OrderController --resource
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Order;
class OrderController extends Controller
{
    public function index()
    {
        $orders = Order::all();
        return view('orders.index', compact('orders'));
    }
    public function create()
    {
        return view('orders.create');
    }
    public function store(Request $request)
    {
        $request->validate([
            'customer_id' => 'required',
            'product_name' => 'required',
            'quantity' => 'required|integer',
            'price' => 'required|numeric'
        ]);

        Order::create($request->all());
        return redirect('/orders')->with('success', 'Order created successfully!');
    }
}
```

(web.php)

Option 1: Resource Route (Recommended)

```
use App\Http\Controllers\OrderController;

Route::resource('orders', OrderController::class)->only([
    'index', 'create', 'store'
]);
```

Option 2: Manual Routes

```
Route::get('/orders', [OrderController::class, 'index']);
Route::get('/orders/create', [OrderController::class, 'create']);
Route::post('/orders', [OrderController::class, 'store'])->name('orders.store');
```

Mass No. 6

Design a responsive layout with 2 columns on desktop and 1 column on mobile. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Layout</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-6">\
    <!-- Grid Container -->
    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <!-- Column 1 -->
        <div class="bg-white p-6 rounded-lg shadow-md">
            <h2 class="text-xl font-bold mb-2">Column 1</h2>
            <p>This column takes full width on mobile and half width on desktop.</p>
        </div>
        <!-- Column 2 -->
        <div class="bg-white p-6 rounded-lg shadow-md">
            <h2 class="text-xl font-bold mb-2">Column 2</h2>
            <p>This is the second column with responsive behavior.</p>
        </div>
    </div>
</body>
```

```
</html>
```

• Create a Laravel form to upload a file and store it using public disk. (10M)

(web.php)

```
use App\Http\Controllers\FileController;

Route::get('/file-upload', [FileController::class, 'index']);
Route::post('/file-upload', [FileController::class, 'store'])->name('file.upload');
```

Create Controller

```
php artisan make:controller FileController
```

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class FileController extends Controller

{

    public function index()

    {

        return view('file-upload');

    }

    public function store(Request $request)

    {

        $request->validate([

            'file' => 'required|file|max:2048'

        ]);


        $path = $request->file('file')->store('uploads', 'public');

        return back()->with('success', 'File uploaded successfully!')

                    ->with('file', $path);

    }

}
```

Create Storage Link

```
php artisan storage:link
```

(resources/views/file-upload.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>File Upload</title>

</head>

<body>

    <h2>Upload File</h2>
```

```
    <!-- Success Message -->
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <!-- Form -->
    <form action="{{ route('file.upload') }}" method="POST" enctype="multipart/form-data">
        @csrf
        <input type="file" name="file">
        @error('file')
            <p style="color:red;">{{ $message }}</p>
        @enderror
        <br><br>
        <button type="submit">Upload</button>
    </form>
    <!-- Display File Link -->
    @if(session('file'))
        <p>File Path: {{ asset('storage/' . session('file')) }}</p>
    @endif
</body></html>
```

• Create a Controller to delete a record from Customer (id, name, contact_no) table using ID. (10M)

php artisan make:model Customer

(app/Models/Customer.php)

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Customer extends Model
{
    protected $fillable = ['name', 'contact_no'];
}
```

Create Controller

php artisan make:controller CustomerController

```
namespace App\Http\Controllers;

use App\Models\Customer;

class CustomerController extends Controller

{

    public function destroy($id)

    {

        $customer = Customer::findOrFail($id);
```

```
        $customer->delete();
        return redirect('/customers')->with('success', 'Customer deleted successfully!');
    }
}
```

Simple Route(web.php)

```
use App\Http\Controllers\CustomerController;

Route::get('/customers/delete/{id}', [CustomerController::class, 'destroy']);
```

Mass No. 7

•Create a navigation bar using Tailwind CSS with 4 menu items aligned horizontally. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Navbar</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body>
    <!-- Navbar -->
    <nav class="bg-blue-600 p-4">
        <div class="flex justify-center space-x-8 text-white font-semibold">
            <a href="#" class="hover:text-gray-200">Home</a>
            <a href="#" class="hover:text-gray-200">About</a>
            <a href="#" class="hover:text-gray-200">Services</a>
            <a href="#" class="hover:text-gray-200">Contact</a>
        </div>
    </nav>
</body>
</html>
```

• Fetch all records from the orders table and display them in a Blade view in tabular format. (10M)

```
php artisan make:controller OrderController
namespace App\Http\Controllers;
use App\Models\Order;
```

```
class OrderController extends Controller
{
    public function index()
    {
        $orders = Order::all(); // Get all records
        return view('orders.index', compact('orders'));    }    }
```

(web.php)

```
use App\Http\Controllers\OrderController;

Route::get('/orders', [OrderController::class, 'index']);
```

(resources/views/orders/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Orders List</title>
</head>
<body>
    <h2>All Orders</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Customer ID</th>
            <th>Product Name</th>
            <th>Quantity</th>
            <th>Price</th>
        </tr>
        @forelse($orders as $order)
        <tr>
            <td>{{ $order->id }}</td>
            <td>{{ $order->customer_id }}</td>
            <td>{{ $order->product_name }}</td>
            <td>{{ $order->quantity }}</td>
            <td>{{ $order->price }}</td>
        </tr>
        @empty
        <tr>
            <td colspan="5">No orders found</td>
```

```
        </tr>
        @endforelse
    </table>
</body>
</html>
```

• Create a Seeder to insert sample data (at least 500 records) into the orders table. (10M)

Create Seeder

php artisan make:seeder OrderSeeder

code:->

```
namespace Database\Seeders;
use Illuminate\Database\Seeder;
use App\Models\Order;
class OrderSeeder extends Seeder
{
    public function run()
    {
        for ($i = 1; $i <= 500; $i++) {
            Order::create([
                'customer_id' => rand(1, 100),
                'product_name' => 'Product ' . $i,
                'quantity' => rand(1, 10),
                'price' => rand(100, 1000),
            ]);
        }
    }
}
```

Register Seeder

Open database/seeders/DatabaseSeeder.php:

```
public function run()
{
    $this->call(OrderSeeder::class);
}
```

Run Seeder

php artisan db:seed

Mass No. 8

• Display an image with fixed size, rounded corners, and shadow using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Image Design</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
   <!-- Image -->
   <img  src="https://via.placeholder.com/300"alt="Sample Image"
      class="w-64 h-64 object-cover rounded-xl shadow-lg">
</body>
</html>
```

• Create a Laravel form with file upload and validate file type (image only). (10M)

(web.php)

```
use App\Http\Controllers\UploadController;

Route::get('/upload-image', [UploadController::class, 'index']);
Route::post('/upload-image', [UploadController::class, 'store'])->name('image.store');
```

Create Controller

```
php artisan make:controller UploadController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UploadController extends Controller
{
   public function index()
   {
      return view('upload-image');
   }
   public function store(Request $request)
   {
      $request->validate([
```

```
        'image' => 'required|image|mimes:jpg,jpeg,png,gif|max:2048'
    ]);
    $path = $request->file('image')->store('images', 'public');

    return back()->with('success', 'Image uploaded successfully!')
                ->with('image', $path);
  }
}
```

. Create Storage Link

```
php artisan storage:link
```

(resources/views/upload-image.blade.php)

```
<!DOCTYPE html>
<html>
<head>
  <title>Upload Image</title>
</head>
<body>
  <h2>Upload Image</h2>
  <!-- Success Message -->
  @if(session('success'))
    <p style="color: green;">{{ session('success') }}</p>
  @endif
  <form action="{{ route('image.store') }}" method="POST" enctype="multipart/form-data">
    @csrf
    <input type="file" name="image">
    @error('image')
      <p style="color:red;">{{ $message }}</p>
    @enderror
    <br><br>
    <button type="submit">Upload</button>
  </form>

  <!-- Display Uploaded Image -->
  @if(session('image'))
    <h3>Preview:</h3>
```

```
        <img src="{{ asset('storage/' . session('image')) }}" width="200">
    @endif
</body></html>
```

• Run Seeder and display inserted records from the orders table. (10M)

```
php artisan db:seed

namespace App\Http\Controllers;
use App\Models\Order;
class OrderController extends Controller
{
    public function index()
    {
        $orders = Order::all(); // Fetch all seeded data
        return view('orders.index', compact('orders'));
    }
}
```

(web.php)

```
use App\Http\Controllers\OrderController;
Route::get('/orders', [OrderController::class, 'index']);
```

View (display.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Orders Data</title>
</head>
<body>
    <h2>Seeded Orders List</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Customer ID</th>
            <th>Product</th>
            <th>Quantity</th>
            <th>Price</th>
```

```
        </tr>
        @foreach($orders as $order)
        <tr>
            <td>{{ $order->id }}</td>
            <td>{{ $order->customer_id }}</td>
            <td>{{ $order->product_name }}</td>
            <td>{{ $order->quantity }}</td>
            <td>{{ $order->price }}</td>
        </tr>
        @endforeach
    </table>
</body>
</html>
```

Mass No. 9

• Create a list with equal vertical spacing and background color using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tailwind List</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <!-- List Container -->
    <ul class="bg-white p-6 rounded-lg shadow-md space-y-4 w-64">
        <li class="bg-blue-500 text-white p-3 rounded">
            Item 1
        </li>
        <li class="bg-blue-500 text-white p-3 rounded">
            Item 2
        </li>
        <li class="bg-blue-500 text-white p-3 rounded">
            Item 3
```

```
        </li>
        <li class="bg-blue-500 text-white p-3 rounded">
            Item 4
        </li>
    </ul>
</body>
</html>
```

• Fetch records from products table where price > 1000 and display them. (10M)

php artisan make:controller ProductController

```
namespace App\Http\Controllers;
use App\Models\Product;
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::where('price', '>', 1000)->get();
        return view('products.index', compact('products'));
    }
}
```

Route (web.php)

```
use App\Http\Controllers\ProductController;
Route::get('/products', [ProductController::class, 'index']);
```

(resources/views/products/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Products List</title>
</head>
<body>
    <h2>Products (Price > 1000)</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
```

```
        </tr>
        @forelse($products as $product)
        <tr>
            <td>{{ $product->id }}</td>
            <td>{{ $product->name }}</td>
            <td>{{ $product->price }}</td>
        </tr>
        @empty
        <tr>
            <td colspan="3">No products found</td>
        </tr>
        @endforelse

    </table>
</body>
</html>
```

• Develop a simple CRUD (Create & Read) for categories table. (10M)

Create Model + Migration

```
php artisan make:model Category -m
public function up()
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->timestamps();
    });
}
php artisan migrate
```

(app/Models/Category.php)

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    protected $fillable = ['name'];
}
```

Create Controller

```
php artisan make:controller CategoryController
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Category;
class CategoryController extends Controller
{
    public function index()
    {
        $categories = Category::all();
        return view('categories.index', compact('categories'));
    }
    public function create()
    {
        return view('categories.create');
    }
    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|min:3'
        ]);
        Category::create([
            'name' => $request->name
        ]);
        return redirect('/categories')->with('success', 'Category added successfully!');
    }
}
```

(web.php)

```
use App\Http\Controllers\CategoryController;

Route::get('/categories', [CategoryController::class, 'index']);
Route::get('/categories/create', [CategoryController::class, 'create']);
Route::post('/categories', [CategoryController::class, 'store'])->name('categories.store');
```

resources/views/categories/create.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Category</title>
```

```
</head>
<body>
    <h2>Add Category</h2>
    <form action="{{ route('categories.store') }}" method="POST">
        @csrf
        <input type="text" name="name" placeholder="Category Name">
        @error('name')
            <p style="color:red;">{{ $message }}</p>
        @enderror
        <br><br>
        <button type="submit">Add</button>
    </form>
</body>
</html>
```

resources/views/categories/index.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Categories</title>
</head>
<body>
    <h2>All Categories</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <a href="/categories/create">Add Category</a>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
        </tr>
        @foreach($categories as $cat)
        <tr>
            <td>{{ $cat->id }}</td>
            <td>{{ $cat->name }}</td>
        </tr>
        @endforeach
```

```
    </table>
</body>
</html>
```

Mass No. 10

• Design a footer with centered text, padding, and background color. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Footer</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="flex flex-col min-h-screen">
    <!-- Content -->
    <div class="flex-grow flex items-center justify-center">
        <h1 class="text-2xl">Main Content</h1>
    </div>
    <!-- Footer -->
    <footer class="bg-gray-800 text-white text-center p-4">
        2026 My Website. All rights reserved.
    </footer></body></html>
```

• Create a migration for categories table with name and description fields. (10M)

Create Migration

```
php artisan make:migration create_categories_table
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
    public function up()
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
```

```
            $table->string('name');
            $table->text('description')->nullable();
            $table->timestamps();
        });
    }
    public function down()
    {
        Schema::dropIfExists('categories');
    }
};
```

php artisan migrate

• Create a Model with $fillable property and insert data into products table. (10M)

Create Model

php artisan make:model Product

app/Models/Product.php:

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $fillable = [
        'name',
        'price'
    ];
}
```

Create Controller

php artisan make:controller ProductController

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Product;
class ProductController extends Controller
{
    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required',
            'price' => 'required|numeric'
        ]);
        Product::create([
```

```
            'name' => $request->name,
            'price' => $request->price
        ]);
        return back()->with('success', 'Product added successfully!');
    }
}
```

(web.php)

```
use App\Http\Controllers\ProductController;

Route::post('/products', [ProductController::class, 'store'])->name('products.store');
```

Mass No. 11

• Create a div using Tailwind CSS with border, padding, and margin utilities. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Tailwind Div</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100">
    <!-- Div -->
    <div class="border border-gray-400 p-6 m-6 bg-white rounded-lg shadow-md">
        <h2 class="text-xl font-bold mb-2">Sample Div</h2>
        <p>This div uses Tailwind CSS border, padding, and margin utilities.</p>
    </div></body></html>
```

• Insert multiple records into students table using Eloquent ORM. (10M)

Create Model + Migration

Run this command:

```
php artisan make:model Student -m
```

Go to folder:
database/migrations

Open file: create_students_table.php

```
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->timestamps();
    });
}
```

```
php artisan migrate
```

Go to:
app/Models/Student.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = ['name', 'email'];
}
```

Create Controller

Run:

php artisan make:controller StudentController

```
namespace App\Http\Controllers;

use App\Models\Student;

class StudentController extends Controller

{

    public function insert()

    {

        $data = [

            ['name' => 'Tejas', 'email' => 'tejas@gmail.com'],

            ['name' => 'Rahul', 'email' => 'rahul@gmail.com'],

            ['name' => 'Amit', 'email' => 'amit@gmail.com']

        ];

        Student::insert($data);

        return "Inserted successfully";

    }

}
```

(routes/web.php)

```
use App\Http\Controllers\StudentController;

Route::get('/insert-students', [StudentController::class, 'insert']);
```

• Create a Controller to fetch first 5 records from users table. (10M)

php artisan make:controller UserController

Open UserController.php and write:

```
namespace App\Http\Controllers;
use App\Models\User;
class UserController extends Controller
{
   public function index()
   {
      $users = User::take(5)->get();
      return view('users.index', compact('users'));
   }
}
```

(routes/web.php)

```
use App\Http\Controllers\UserController;

Route::get('/users', [UserController::class, 'index']);
```

Create file:
resources/views/users/index.blade.php

```
<!DOCTYPE html>

<html>

<head>

   <title>Users</title>

</head>

<body>

   <h2>First 5 Users</h2>

   <table border="1" cellpadding="10">

      <tr>

         <th>ID</th>

         <th>Name</th>

         <th>Email</th>

      </tr>

      @foreach($users as $user)

      <tr>

         <td>{{ $user->id }}</td>

         <td>{{ $user->name }}</td>

         <td>{{ $user->email }}</td>

      </tr>

      @endforeach

   </table></body></html>
```

Mass No. 12

• Design a centered container using Tailwind CSS with max-width and mx-auto. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Centered Container</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100">
   <!-- Centered Container -->
   <div class="max-w-4xl mx-auto bg-white p-6 mt-10 rounded-lg shadow-md">
      <h2 class="text-2xl font-bold mb-4">Centered Container</h2>
      <p class="text-gray-700">
         This container is centered using Tailwind CSS.
         It uses max-width and mx-auto for proper alignment.
      </p>
   </div>
</body>
</html>
```

• Create a Laravel form and store submitted data into users table. (10M)

Create Controller

```
php artisan make:controller UserControlle
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\User;
class UserController extends Controller
{
   public function create()
   {
      return view('users.create');
   }
   public function store(Request $request)
```

```
    {
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email'
        ]);
        User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => bcrypt('123456')
        ]);
        return redirect('/users/create')->with('success', 'User added successfully!');
    }
}
```

app/Models/User.php

Make sure this is present:

```
protected $fillable = ['name', 'email', 'password'];
```

(resources/views/users/create.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Add User</title>
</head>
<body>
    <h2>Add User</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ url('/users') }}" method="POST">
        @csrf
        <input type="text" name="name" placeholder="Enter Name"><br><br>
        <input type="email" name="email" placeholder="Enter Email"><br><br>
        @error('name')
            <p style="color:red;">{{ $message }}</p>
        @enderror
        @error('email')
            <p style="color:red;">{{ $message }}</p>
        @enderror
```

```
        <button type="submit">Submit</button>
    </form></body></html>
```

routes/web.php

```
use App\Http\Controllers\UserController;

Route::get('/users/create', [UserController::class, 'create']);
Route::post('/users', [UserController::class, 'store']);
```

• Create a Controller to count total records from students table and display the result. (10M)

Create Controller

```
php artisan make:controller StudentController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Student;
class StudentController extends Controller
{
    public function countStudents()
    {
        $total = Student::count();

        return view('students.count', compact('total'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\StudentController;
Route::get('/students/count', [StudentController::class, 'countStudents']);
```

(resources/views/students/count.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>Student Count</title>

</head>

<body>

    <h2>Total Students</h2>

    <h3>{{ $total }}</h3>

</body>

</html>
```

Mass No. 13

• Create a responsive heading using Tailwind CSS with different font sizes for mobile and desktop. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Responsive Heading</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
   <!-- Responsive Heading -->
   <h1 class="text-xl md:text-4xl font-bold text-blue-600">
      Responsive Tailwind Heading
   </h1>
</body></html>
```

• Create a Laravel form with validation and redirect back with old input values. (10M)

Create Controller

```
php artisan make:controller FormController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class FormController extends Controller
{
   public function showForm()
   {
      return view('form');
   }
   public function submitForm(Request $request)
   {
      $request->validate([
         'name' => 'required|min:3',
         'email' => 'required|email'
      ]);

      return back()->with('success', 'Form submitted successfully!');
   }
}
```

(routes/web.php)

```
use App\Http\Controllers\FormController;
```

```
Route::get('/form', [FormController::class, 'showForm']);
Route::post('/form', [FormController::class, 'submitForm'])->name('form.submit');
```

(resources/views/form.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>Form</title>

</head>

<body>

    <h2>Form</h2>

    @if(session('success'))

        <p style="color: green;">{{ session('success') }}</p>

    @endif

    <form action="{{ route('form.submit') }}" method="POST">

        @csrf

        <input type="text" name="name" value="{{ old('name') }}" placeholder="Enter
Name"><br><br>

        <input type="text" name="email" value="{{ old('email') }}" placeholder="Enter
Email"><br><br>

        @error('name')

            <p style="color:red;">{{ $message }}</p>

        @enderror

        @error('email')

            <p style="color:red;">{{ $message }}</p>

        @enderror


        <button type="submit">Submit</button>

    </form></body></html>
```

• Create a Controller and pass data to Blade view using compact() method from employees table. (10M)

```
php artisan make:model Employee

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Employee extends Model

{

    protected $fillable = ['name', 'email', 'salary'];

}
```

Create Controller

```
php artisan make:controller EmployeeController
```

```
namespace App\Http\Controllers;
use App\Models\Employee;
class EmployeeController extends Controller
{
    public function index()
    {
        $employees = Employee::all();
        return view('employees.index', compact('employees'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\EmployeeController;
Route::get('/employees', [EmployeeController::class, 'index']);
```

(resources/views/employees/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Employees</title>
</head>
<body>
    <h2>Employee List</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Salary</th>
        </tr>
        @foreach($employees as $emp)
        <tr>
            <td>{{ $emp->id }}</td>
            <td>{{ $emp->name }}</td>
```

```
            <td>{{ $emp->email }}</td>
            <td>{{ $emp->salary }}</td>
        </tr>
        @endforeach
    </table></body></html>
```

Mass No. 14

• Design a button group with 3 buttons placed side by side using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Button Group</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <!-- Button Group -->
    <div class="flex">
        <button class="bg-blue-500 text-white px-4 py-2 rounded-l-lg hover:bg-blue-600">
            Left
        </button>
<button class="bg-blue-500 text-white px-4 py-2 border-l border-white hover:bg-blue-600">
            Middle
        </button>
        <button class="bg-blue-500 text-white px-4 py-2 rounded-r-lg border-l border-white hover:bg-
blue-600">
            Right
        </button>
    </div></body></html>
```

• Update a record in Books table using Eloquent ORM. (10M)

```
php artisan make:model Book
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Book extends Model
```

```
{
    protected $fillable = ['title', 'author', 'price'];
}
```

Create Controller

```
php artisan make:controller BookController
```

Controller Code (Update)

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Book;
class BookController extends Controller
{
    public function edit($id)
    {
        $book = Book::findOrFail($id);
        return view('books.edit', compact('book'));
    }
    public function update(Request $request, $id)
    {
        $request->validate([
            'title' => 'required',
            'author' => 'required',
            'price' => 'required|numeric'
        ]);
        $book = Book::findOrFail($id);
        $book->update([
            'title' => $request->title,
            'author' => $request->author,
            'price' => $request->price
        ]);
        return redirect('/books')->with('success', 'Book updated successfully!');
    } }
```

routes/web.php

```
use App\Http\Controllers\BookController;

Route::get('/books/edit/{id}', [BookController::class, 'edit']);
Route::post('/books/update/{id}', [BookController::class, 'update'])->name('books.update');
```

resources/views/books/edit.blade.php

```
<!DOCTYPE html>
```

```
<html>
<head>
    <title>Edit Book</title>
</head>
<body>
    <h2>Edit Book</h2>
    <form action="{{ route('books.update', $book->id) }}" method="POST">
        @csrf
        <input type="text" name="title" value="{{ $book->title }}"><br><br>
        <input type="text" name="author" value="{{ $book->author }}"><br><br>
        <input type="text" name="price" value="{{ $book->price }}"><br><br>
        <button type="submit">Update</button>
    </form></body></html>
```

• Create a Controller to fetch records with condition (e.g., price > 500) from products table. (10M)

Create Controller

```
php artisan make:controller ProductController
```

Controller Code

```
namespace App\Http\Controllers;

use App\Models\Product;

class ProductController extends Controller
{
    public function index()
    {
        $products = Product::where('price', '>', 500)->get();

        return view('products.index', compact('products'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\ProductController;

Route::get('/products', [ProductController::class, 'index']);
```

(resources/views/products/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Products</title>
</head>
<body>
```

```
    <h2>Products (Price > 500)</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
        </tr>
        @forelse($products as $product)
        <tr>
            <td>{{ $product->id }}</td>
            <td>{{ $product->name }}</td>
            <td>{{ $product->price }}</td>
        </tr>
        @empty
        <tr>
            <td colspan="3">No data found</td>
        </tr>
        @endforelse
    </table></body></html>
```

Mass No. 15

• Create a responsive navbar using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Navbar</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <script>
        function toggleMenu() {
            document.getElementById('menu').classList.toggle('hidden');
        }
    </script>
</head>
```

```
<body class="bg-gray-100">
    <!-- Navbar -->
    <nav class="bg-blue-600 p-4 text-white">
        <div class="flex justify-between items-center">
            <!-- Logo -->
            <div class="text-lg font-bold">MySite</div>
            <!-- Hamburger Button -->
            <button class="md:hidden" onclick="toggleMenu()">
            </button>
            <!-- Menu -->
            <div id="menu" class="hidden md:flex space-x-6">
                <a href="#" class="hover:text-gray-200">Home</a>
                <a href="#" class="hover:text-gray-200">About</a>
                <a href="#" class="hover:text-gray-200">Services</a>
                <a href="#" class="hover:text-gray-200">Contact</a>
            </div> </div>
    </nav></body></html>
```

• Create a Laravel form with password and confirm password validation. (10M)

Create Controller

```
php artisan make:controller RegisterController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class RegisterController extends Controller
{
    public function showForm()
    {
        return view('register');
    }

    public function register(Request $request)
    {
        $request->validate([
            'password' => 'required|min:6|confirmed'
        ]);

        return back()->with('success', 'Password matched and validated!');
    }
}
```

routes/web.php

```
use App\Http\Controllers\RegisterController;
```

```
Route::get('/register', [RegisterController::class, 'showForm']);
Route::post('/register', [RegisterController::class, 'register'])->name('register.store');
```

(resources/views/register.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>Register</title>

</head>

<body>


    <h2>Register Form</h2>

    @if(session('success'))

        <p style="color: green;">{{ session('success') }}</p>

    @endif

    <form action="{{ route('register.store') }}" method="POST">

        @csrf

        <!-- Password -->

        <input type="password" name="password" placeholder="Enter Password"><br><br>

        <!-- Confirm Password -->

        <input type="password" name="password_confirmation" placeholder="Confirm
Password"><br><br>

        @error('password')

            <p style="color:red;">{{ $message }}</p>

        @enderror

        <button type="submit">Register</button>

    </form></body></html>
```

• Create a Model and perform insert operation into orders table. (10M)

Create Model

```
php artisan make:model Order
```

app/Models/Order.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Order extends Model
{
    protected $fillable = [
        'customer_id',
        'product_name',
        'quantity',
        'price'
```

```
    ];
}
```

Create Controller

php artisan make:controller OrderController

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Order;
class OrderController extends Controller
{
    public function create()
    {
        return view('orders.create');
    }
    public function store(Request $request)
    {
        $request->validate([
            'customer_id' => 'required',
            'product_name' => 'required',
            'quantity' => 'required|integer',
            'price' => 'required|numeric'
        ]);
        Order::create([
            'customer_id' => $request->customer_id,
            'product_name' => $request->product_name,
            'quantity' => $request->quantity,
            'price' => $request->price
        ]);
        return back()->with('success', 'Order added successfully!');
    }
}
```

(routes/web.php)

```
use App\Http\Controllers\OrderController;

Route::get('/orders/create', [OrderController::class, 'create']);
Route::post('/orders', [OrderController::class, 'store'])->name('orders.store');
```

(resources/views/orders/create.blade.php)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Add Order</title>
</head>
<body>
    <h2>Add Order</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('orders.store') }}" method="POST">
        @csrf
        <input type="text" name="customer_id" placeholder="Customer ID"><br><br>
        <input type="text" name="product_name" placeholder="Product Name"><br><br>
        <input type="text" name="quantity" placeholder="Quantity"><br><br>
        <input type="text" name="price" placeholder="Price"><br><br>
        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

Mass No. 16

• Design a box using Tailwind CSS with gradient background. (5M)

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gradient Box</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <div class="w-80 h-40 bg-gradient-to-r from-blue-500 to-purple-600 text-white
            flex items-center justify-center rounded-xl shadow-lg">
```

```
    <h2 class="text-xl font-bold">Gradient Box</h2>
  </div></body></html>
```

• Create a migration to add a new column, run it, and rollback the last migration using artisan. (10M)

Create Migration

```
php artisan make:migration add_phone_to_users_table --table=users
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
  public function up()
  {
    Schema::table('users', function (Blueprint $table) {
      $table->string('phone')->nullable()->after('email');
    });
  }

  public function down()
  {
    Schema::table('users', function (Blueprint $table) {
      $table->dropColumn('phone');
    });
  }
};
```

Run Migration

```
php artisan migrate
```

Rollback Last Migration

```
php artisan migrate:rollback
```

• Create a Seeder for customers table and insert sample records. (10M)

Create Seeder

```
php artisan make:seeder CustomerSeeder
```

```
namespace Database\Seeders;
```

```
use Illuminate\Database\Seeder;
use App\Models\Customer;
class CustomerSeeder extends Seeder
{
    public function run()
    {
        Customer::insert([
            ['name' => 'Tina', 'contact_no' => '9876543210'],
            ['name' => 'Rahul', 'contact_no' => '9123456780'],
            ['name' => 'Amit', 'contact_no' => '9988776655'],
            ['name' => 'Sneha', 'contact_no' => '9090909090'],
            ['name' => 'Pooja', 'contact_no' => '8888888888']
        ]);
    }
}
```

database/seeders/DatabaseSeeder.php

```
public function run()
{
    $this->call(CustomerSeeder::class);
}
```

Run Seeder

```
php artisan db:seed
```

Mass No. 17

• Create a flex layout with items centered vertically and horizontally using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flex Center</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <!-- Centered Box -->
    <div class="bg-white p-8 rounded-lg shadow-md">
```

```
        <h2 class="text-xl font-bold text-center">Centered Content</h2>
        <p class="text-gray-600 mt-2 text-center">
            This content is perfectly centered using Tailwind Flexbox.
        </p>
    </div>
</body>
</html>
```

• Store uploaded file and generate URL using asset() helper in Laravel. (10M)

routes/web.php

```
use App\Http\Controllers\FileController;

Route::get('/upload', [FileController::class, 'index']);
Route::post('/upload', [FileController::class, 'store'])->name('file.store');
```

Create Controller

```
php artisan make:controller FileController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class FileController extends Controller
{
    public function index()
    {
        return view('upload');
    }
    public function store(Request $request)
    {
        $request->validate([
            'file' => 'required|file'
        ]);
        $path = $request->file('file')->store('uploads', 'public');
        $url = asset('storage/' . $path);
        return back()->with('file_url', $url);
    }
}
```

Create Storage Link

```
php artisan storage:link
```

resources/views/upload.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>File Upload</title>
</head>
<body>
    <h2>Upload File</h2>
    <form action="{{ route('file.store') }}" method="POST" enctype="multipart/form-data">
        @csrf
        <input type="file" name="file"><br><br>
        <button type="submit">Upload</button>
    </form>
    @if(session('file_url'))
        <p>File URL:</p>
        <a href="{{ session('file_url') }}" target="_blank">
            {{ session('file_url') }}
        </a>
    @endif
</body></html>
```

• Create a controller and pass data to Blade using with() method. (10M)

Create Controller

```
php artisan make:controller DemoController
```

Controller Code

```
namespace App\Http\Controllers;
class DemoController extends Controller
{
    public function index()
    {
        $name = "Tina";
        $city = "Pune";
        return view('demo')->with('name', $name)->with('city', $city);
    }
}
```

routes/web.php

```
use App\Http\Controllers\DemoController;

Route::get('/demo', [DemoController::class, 'index']);
```

(resources/views/demo.blade.php)

```
<!DOCTYPE html>
<html>
<head>
   <title>Demo</title>
</head>
<body>
   <h2>Data from Controller</h2>
   <p>Name: {{ $name }}</p>
   <p>City: {{ $city }}</p>
</body>
</html>
```

Mass No. 18

• Create a grid layout with 4 boxes using Tailwind CSS (2 columns). (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Grid Layout</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
   <!-- Grid Container -->
   <div class="grid grid-cols-2 gap-6">
      <!-- Box 1 -->
      <div class="bg-blue-500 text-white p-6 rounded-lg text-center">
         Box 1
      </div>
      <!-- Box 2 -->
      <div class="bg-green-500 text-white p-6 rounded-lg text-center">
         Box 2
      </div>
      <!-- Box 3 -->
      <div class="bg-red-500 text-white p-6 rounded-lg text-center">
         Box 3
      </div>
      <!-- Box 4 -->
      <div class="bg-purple-500 text-white p-6 rounded-lg text-center">
```

```
        Box 4
    </div> </div>
</body></html>
```

• Create a model with $fillable property and insert data into categories table. (10M)

Create Model

```
php artisan make:model Category
```

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Category extends Model
{
    protected $fillable = ['name', 'description'];
}
```

Create Controller

```
php artisan make:controller CategoryController
```

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Category;
class CategoryController extends Controller
{
    public function create()
    {
        return view('categories.create')
    }
    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|min:3',
            'description' => 'required'
        ]);
        Category::create([
            'name' => $request->name,
            'description' => $request->description
        ]);
        return back()->with('success', 'Category added successfully!');
    }
}
```

routes/web.php

```
use App\Http\Controllers\CategoryController;

Route::get('/categories/create', [CategoryController::class, 'create']);
Route::post('/categories', [CategoryController::class, 'store'])->name('categories.store');
```

resources/views/categories/create.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Add Category</title>
</head>
<body>
    <h2>Add Category</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('categories.store') }}" method="POST">
        @csrf
        <input type="text" name="name" placeholder="Category Name"><br><br>
        <textarea name="description" placeholder="Description"></textarea><br><br>
        @error('name')
            <p style="color:red;">{{ $message }}</p>
        @enderror
        @error('description')
            <p style="color:red;">{{ $message }}</p>
        @enderror
        <button type="submit">Submit</button>
    </form>
</body></html>
```

• Insert multiple records into students table using Eloquent ORM. (10M)

Create Model

```
php artisan make:model Student
```

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Student extends Model
```

```
{
    protected $fillable = ['name', 'email'];
}
```

Create Controller

```
php artisan make:controller StudentController
```

```
namespace App\Http\Controllers;
use App\Models\Student;
class StudentController extends Controller
{
    public function insertMultiple()
    {
        $students = [
            ['name' => 'Tejas', 'email' => 'tejas@gmail.com'],
            ['name' => 'Rahul', 'email' => 'rahul@gmail.com'],
            ['name' => 'Amit', 'email' => 'amit@gmail.com'],
            ['name' => 'Sneha', 'email' => 'sneha@gmail.com']
        ];
        Student::insert($students);
        return "Multiple records inserted successfully!";
    }
}
```

routes/web.php

```
use App\Http\Controllers\StudentController;
Route::get('/students/insert', [StudentController::class, 'insertMultiple']);
```

Mass No. 19

• Create a sticky header using Tailwind CSS that remains at the top while scrolling. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sticky Header</title>
    <script src="https://cdn.tailwindcss.com"></script>
```

```
</head>
<body class="bg-gray-100">
    <!-- Header -->
    <header class="sticky top-0 bg-blue-600 text-white p-4 shadow-md z-50">
        <h1 class="text-xl font-bold">Sticky Header</h1>
    </header>
    <!-- Content -->
    <div class="p-6 space-y-6">
        <p>Scroll down to see the sticky header effect.</p>
        <div class="h-[1500px] bg-white rounded shadow p-4">
            Long content here...
        </div>   </div>
</body></html>
```

• Count total records from employees table using Eloquent ORM. (10M)

Create Controller

```
php artisan make:controller EmployeeController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Employee;
class EmployeeController extends Controller
{
    public function count()
    {
        $total = Employee::count();
        return view('employees.count', compact('total'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\EmployeeController;

Route::get('/employees/count', [EmployeeController::class, 'count']);
```

resources/views/employees/count.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Employee Count</title>
</head>
<body>
    <h2>Total Employees</h2>
    <h3>{{ $total }}</h3>
</body></html>
```

• Create a Controller to update a specific record using ID in employees table. (10M)

Create Controller

```
php artisan make:controller EmployeeController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Employee;
class EmployeeController extends Controller
{
    public function edit($id)
    {
        $employee = Employee::findOrFail($id);
        return view('employees.edit', compact('employee'));
    }
    public function update(Request $request, $id)
    {
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email',
            'salary' => 'required|numeric'
        ]);
        $employee = Employee::findOrFail($id);

        $employee->update([
            'name' => $request->name,
            'email' => $request->email,
            'salary' => $request->salary
        ]);
        return redirect('/employees')->with('success', 'Employee updated successfully!');
    }
}
```

routes/web.php

```
use App\Http\Controllers\EmployeeController;

Route::get('/employees/edit/{id}', [EmployeeController::class, 'edit']);
Route::post('/employees/update/{id}', [EmployeeController::class, 'update'])-
>name('employees.update');
```

resources/views/employees/edit.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Edit Employee</title>
</head>
<body>
    <h2>Edit Employee</h2>
    <form action="{{ route('employees.update', $employee->id) }}" method="POST">
        @csrf
        <input type="text" name="name" value="{{ $employee->name }}"><br><br>
        <input type="text" name="email" value="{{ $employee->email }}"><br><br>
        <input type="text" name="salary" value="{{ $employee->salary }}"><br><br>
```

```
    <button type="submit">Update</button>
  </form></body></html>
```

Mass No. 20

• Design a card with hover shadow effect using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hover Card</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">

  <!-- Card -->
  <div class="bg-white p-6 rounded-xl shadow-md hover:shadow-2xl transition duration-300 w-80">
    <h2 class="text-xl font-bold mb-2">Card Title</h2>
    <p class="text-gray-600">
      This card has a hover shadow effect using Tailwind CSS.
    </p></div>
</body></html>
```

• Create a form with multiple fields and validate all inputs in Laravel. (10M)

Create Controller

```
php artisan make:controller FormController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class FormController extends Controller
{
  public function showForm()
  {
    return view('form');
  }
  public function submitForm(Request $request)
  {
    $request->validate([
      'name' => 'required|min:3',
```

```
            'email' => 'required|email',
            'phone' => 'required|digits:10',
            'password' => 'required|min:6|confirmed'
        ]);
        return back()->with('success', 'Form submitted successfully!');
    }
}
```

(routes/web.php)

```
use App\Http\Controllers\FormController;

Route::get('/form', [FormController::class, 'showForm']);
Route::post('/form', [FormController::class, 'submitForm'])->name('form.submit');
```

(resources/views/form.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>Form</title>

</head>

<body>

    <h2>Registration Form</h2>

    @if(session('success'))

        <p style="color: green;">{{ session('success') }}</p>

    @endif

    @if ($errors->any())

        <ul style="color:red;">

            @foreach ($errors->all() as $error)

                <li>{{ $error }}</li>

            @endforeach

        </ul>

    @endif

    <form action="{{ route('form.submit') }}" method="POST">

        @csrf

 <input type="text" name="name" value="{{ old('name') }}" placeholder="Name"><br><br> <input
type="email" name="email" value="{{ old('email') }}"placeholder="Email"><br><br>

  <input type="text" name="phone" value="{{ old('phone') }}" placeholder="Phone"><br><br>

        <input type="password" name="password" placeholder="Password"><br><br>

        <input type="password" name="password_confirmation" placeholder="Confirm
Password"><br><br>

  <button type="submit">Submit</button>

    </form></body></html>
```

• Create a one-to-many relationship between Customer and Orders; fetch and display in Blade. (10M)

php artisan make:model Customer -m

```
public function up()
{
    Schema::create('customers', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('contact_no');
        $table->timestamps();
    });
}
```

php artisan make:model Order -m

```
public function up()
{
    Schema::create('orders', function (Blueprint $table) {
        $table->id();
        $table->unsignedBigInteger('customer_id');
        $table->string('product_name');
        $table->integer('quantity');
        $table->decimal('price', 10, 2);
        $table->timestamps();
        $table->foreign('customer_id')->references('id')->on('customers')->onDelete('cascade');
    });
}
```

Run Migration

```
php artisan migrate
```

app/Models/Customer.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Customer extends Model
{
    protected $fillable = ['name', 'contact_no'];
    public function orders()
    {
        return $this->hasMany(Order::class);
```

```
    }
}
```

app/Models/Order.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Order extends Model
{
    protected $fillable = ['customer_id', 'product_name', 'quantity', 'price'];
   public function customer()
    {
        return $this->belongsTo(Customer::class);
    }
}
```

Create Controller

```
php artisan make:controller CustomerController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Customer;
class CustomerController extends Controller
{
    public function index()
    {
        $customers = Customer::with('orders')->get();
        return view('customers.index', compact('customers'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\CustomerController;

Route::get('/customers', [CustomerController::class, 'index']);
```

(resources/views/customers/index.blade.php)

```
<!DOCTYPE html>

<html>

<head>

    <title>Customers</title>

</head>

<body>

    <h2>Customers and Orders</h2>

    @foreach($customers as $customer)

        <h3>{{ $customer->name }}</h3>

        <ul>

            @forelse($customer->orders as $order)

                <li>

                    {{ $order->product_name }} - {{ $order->quantity }} - {{ $order->price }}
```

```
            </li>
        @empty
            <li>No orders</li>
        @endforelse
    </ul>
    <hr>
  @endforeach
</body></html>
```

Mass No. 21

• Create a progress bar using Tailwind CSS showing 70% completion. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Progress Bar</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
  <div class="w-80">
    <p class="mb-2 font-semibold">Progress: 70%</p>
    <div class="w-full bg-gray-300 rounded-full h-4">
      <div class="bg-blue-500 h-4 rounded-full w-[70%]"></div>
    </div>
  </div>
</body></html>
```

• Fetch limited records (first 5) from orders table using Eloquent. (10M)

Create Controller

```
php artisan make:controller OrderController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Order;
class OrderController extends Controller
{
  public function index()
```

```
    {
        $orders = Order::take(5)->get();
        return view('orders.index', compact('orders'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\OrderController;

Route::get('/orders', [OrderController::class, 'index']);
```

(resources/views/orders/index.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Orders</title>
</head>
<body>
    <h2>First 5 Orders</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Customer ID</th>
            <th>Product</th>
            <th>Quantity</th>
            <th>Price</th>
        </tr>
        @foreach($orders as $order)
        <tr>
            <td>{{ $order->id }}</td>
            <td>{{ $order->customer_id }}</td>
            <td>{{ $order->product_name }}</td>
            <td>{{ $order->quantity }}</td>
            <td>{{ $order->price }}</td>
        </tr>
        @endforeach
    </table>
</body></html>
```

• Implement soft delete in users table and retrieve deleted records. (10M)

Create migration:

php artisan make:migration add_deleted_at_to_users_table --table=users

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
    public function up()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->softDeletes();
        });
    }
    public function down()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropSoftDeletes();
        });
    }
};
```

php artisan migrate

app/Models/User.php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
class User extends Model
{
    use SoftDeletes;
    protected $fillable = ['name', 'email', 'password'];
}
```

Create Controller

```
php artisan make:controller UserController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\User;
class UserController extends Controller
{
```

```
    public function delete($id)
    {
        $user = User::findOrFail($id);
        $user->delete();
        return "User soft deleted";
    }
    public function trashed()
    {
        $users = User::onlyTrashed()->get();
        return view('users.trashed', compact('users'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\UserController;

Route::get('/users/delete/{id}', [UserController::class, 'delete']);
Route::get('/users/trashed', [UserController::class, 'trashed']);
```

resources/views/users/trashed.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Deleted Users</title>
</head>
<body>
    <h2>Soft Deleted Users</h2>
    <ul>
        @foreach($users as $user)
            <li>{{ $user->name }} - {{ $user->email }}</li>
        @endforeach
    </ul>
</body></html>
```

Mass No. 22

• Design a semi-transparent box using opacity utilities in Tailwind CSS. (5M)

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Opacity Box</title>

    <script src="https://cdn.tailwindcss.com"></script>

</head>

<body class="bg-[url('https://via.placeholder.com/800')] bg-cover h-screen flex items-center justify-center">

    <div class="bg-black bg-opacity-50 text-white p-8 rounded-lg shadow-lg">

        <h2 class="text-xl font-bold mb-2">Transparent Box</h2>
```

```
        <p>This box has 50% opacity using Tailwind CSS.</p>
    </div></body></html>
```

• Check if a record exists in products table using Eloquent ORM. (10M)

Create Controller

```
php artisan make:controller ProductController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Product;
class ProductController extends Controller
{
    public function check()
    {
        $exists = Product::where('price', '>', 1000)->exists();
        if ($exists) {
            return "Record exists";
        } else {
            return "No record found";
        }
    }
}
```

routes/web.php

```
use App\Http\Controllers\ProductController;
Route::get('/products/check', [ProductController::class, 'check']);
```

• Create API route and return JSON response from controller. (10M)

Create Controller

```
php artisan make:controller Api/ProductControlle
```

Controller Code

```
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use App\Models\Product;
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return response()->json([
            'status' => true,
            'data' => $products
        ]);
    }
}
```

routes/api.php

```
use App\Http\Controllers\Api\ProductController;

Route::get('/products', [ProductController::class, 'index']);
```

Mass No. 23

• Create a dropdown menu visible on hover using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Dropdown Hover</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
   <div class="relative group">
      <button class="bg-blue-500 text-white px-4 py-2 rounded">
         Menu
      </button>
    <div class="absolute hidden group-hover:block bg-white shadow-lg rounded mt-2 w-40">
         <a href="#" class="block px-4 py-2 hover:bg-gray-200">Profile</a>
         <a href="#" class="block px-4 py-2 hover:bg-gray-200">Settings</a>
         <a href="#" class="block px-4 py-2 hover:bg-gray-200">Logout</a>
      </div> </div>
</body></html>
```

• Create a migration and drop a column from users table. (10M)

Create Migration

```
php artisan make:migration drop_phone_from_users_table --table=users
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
   public function up()
   {
      Schema::table('users', function (Blueprint $table) {
         $table->dropColumn('phone');
      });
```

```
    }
    public function down()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('phone')->nullable();
        });
    }
};
```

Run Migration

```
php artisan migrate
```

• Count total records from orders table using Eloquent ORM. (10M)

Create Controller

```
php artisan make:controller OrderController
```

Controller Code

```
namespace App\Http\Controllers;
use App\Models\Order;
class OrderController extends Controller
{
    public function count()
    {
        $total = Order::count();
        return view('orders.count', compact('total'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\OrderController;

Route::get('/orders/count', [OrderController::class, 'count']);
```

resources/views/orders/count.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Orders Count</title>
</head>
<body>
    <h2>Total Orders</h2>
    <h3>{{ $total }}</h3>
</body></html>
```

Mass No. 24

• Design a search input box with focus effect using Tailwind CSS. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Search Box</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
   <div class="w-80">
      <input
         type="text"
         placeholder="Search..."
         class="w-full px-4 py-2 border rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-blue-500 transition duration-300">
</div></body></html>
```

• Create a form to upload file and store it with custom filename in Laravel. (10M)

Create Controller

```
php artisan make:controller FileUploadController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class FileUploadController extends Controller
{
   public function index()
   {
      return view('upload');
   }
   public function store(Request $request)
   {
      $request->validate([
         'file' => 'required|file'
      ]);
      $file = $request->file('file');
      $filename = time() . '_' . $file->getClientOriginalName();

      $path = $file->storeAs('uploads', $filename, 'public');
      return back()->with('success', 'File uploaded successfully!')
               ->with('file', $path);
```

```
    }
}
```

routes/web.php

```
use App\Http\Controllers\FileUploadController;

Route::get('/upload', [FileUploadController::class, 'index']);
Route::post('/upload', [FileUploadController::class, 'store'])->name('file.upload');
```

Create Storage Link

```
php artisan storage:link
```

esources/views/upload.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Upload File</title>
</head>
<body>
    <h2>Upload File</h2>
    @if(session('success'))
        <p style="color: green;">{{ session('success') }}</p>
    @endif
    <form action="{{ route('file.upload') }}" method="POST" enctype="multipart/form-data">
        @csrf
        <input type="file" name="file"><br><br>
        <button type="submit">Upload</button>
    </form>
    @if(session('file'))
        <p>Stored Path: {{ session('file') }}</p>
    @endif
</body></html>
```

• Create Model and fetch first 5 records from customers table using Eloquent ORM. (10M)

Create Model

```
php artisan make:model Customer
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Customer extends Model

{

protected $fillable = ['name', 'contact_no'];

}
```

Create Controller

```
php artisan make:controller CustomerController

namespace App\Http\Controllers;

use App\Models\Customer;

class CustomerController extends Controller
```

```
{
    public function index()
    {
        $customers = Customer::take(5)->get();
        return view('customers.index', compact('customers'));
    }
}
```

routes/web.php

```
use App\Http\Controllers\CustomerController;

Route::get('/customers', [CustomerController::class, 'index']);
```

resources/views/customers/index.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Customers</title>
</head>
<body>
    <h2>First 5 Customers</h2>
    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Contact</th>
        </tr>
  @foreach($customers as $customer)
        <tr>
            <td>{{ $customer->id }}</td>
            <td>{{ $customer->name }}</td>
            <td>{{ $customer->contact_no }}</td>
        </tr>
        @endforeach
    </table></body></html>
```

Mass No. 25

• Create a box using Tailwind CSS that is hidden on mobile and visible on desktop. (5M)

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Responsive Box</title>

    <script src="https://cdn.tailwindcss.com"></script>
```

```
</head>
<body class="bg-gray-100 flex items-center justify-center h-screen">
    <div class="hidden md:block bg-blue-500 text-white p-6 rounded-lg shadow-lg">
        Visible only on Desktop
    </div></body></html>
```

• Create a form with validation and store data into students table. (10M)

Create Model + Migration

```
php artisan make:model Student -m
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->timestamps();
    });
}
php artisan migrate
```

app/Models/Student.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Student extends Model
{
    protected $fillable = ['name', 'email'];
}
```

app/Models/Student.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Student extends Model
{
    protected $fillable = ['name', 'email'];
}
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Student;
class StudentController extends Controller
{
    public function create()
    {
        return view('students.create');
    }
    public function store(Request $request)
    {
```

```
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email'
        ]);
        Student::create([
            'name' => $request->name,
            'email' => $request->email
        ]);
        return back()->with('success', 'Student added successfully!');
    }
}
```

routes/web.php

```
use App\Http\Controllers\StudentController;

Route::get('/students/create', [StudentController::class, 'create']);
Route::post('/students', [StudentController::class, 'store'])->name('students.store');
```

resources/views/students/create.blade.php

```
<!DOCTYPE html>

<html>

<head>

    <title>Add Student</title>

</head>

<body>

    <h2>Add Student</h2>

    @if(session('success'))

        <p style="color: green;">{{ session('success') }}</p>

    @endif

    <form action="{{ route('students.store') }}" method="POST">

        @csrf

        <input type="text" name="name" value="{{ old('name') }}" placeholder="Name"><br><br>

        <input type="email" name="email" value="{{ old('email') }}" placeholder="Email"><br><br>

        @error('name')

            <p style="color:red;">{{ $message }}</p>

        @enderror

        @error('email')

            <p style="color:red;">{{ $message }}</p>

        @enderro

        <button type="submit">Submit</button>

    </form>

</body></html>
```

• Develop a complete CRUD application for students table using Model and Controller. (10M)

Create Model + Migration

```
php artisan make:model Student -m


public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email');
        $table->timestamps();
    });
}
```

```
php artisan migrate
```

app/Models/Student.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Student extends Model
{
    protected $fillable = ['name', 'email'];
}
```

Create Controller

```
php artisan make:controller StudentController
```

Controller Code (FULL CRUD)

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Student;
class StudentController extends Controller
{
    public function index()
    {
        $students = Student::all();
        return view('students.index', compact('students'));
    }
    public function create()
    {
        return view('students.create');
    }
    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email'
        ]);
        Student::create($request->all());
```

```
        return redirect('/students')->with('success', 'Student added');
    }

    public function edit($id)
    {
        $student = Student::findOrFail($id);
        return view('students.edit', compact('student'));
    }
    public function update(Request $request, $id)
    {
        $request->validate([
            'name' => 'required',
            'email' => 'required|email'
        ]);

        $student = Student::findOrFail($id);
        $student->update($request->all());
        return redirect('/students')->with('success', 'Student updated');
    }

    public function destroy($id)
    {
        Student::findOrFail($id)->delete();
        return redirect('/students')->with('success', 'Student deleted');
    }
}
```

routes/web.php

```
use App\Http\Controllers\StudentController;

Route::get('/students', [StudentController::class, 'index']);
Route::get('/students/create', [StudentController::class, 'create']);
Route::post('/students', [StudentController::class, 'store'])->name('students.store');
Route::get('/students/edit/{id}', [StudentController::class, 'edit']);
Route::post('/students/update/{id}', [StudentController::class, 'update'])->name('students.update');
Route::get('/students/delete/{id}', [StudentController::class, 'destroy']);
```

(resources/views/students/index.blade.php)

```
<h2>Students</h2>

<a href="/students/create">Add Student</a>

<table border="1">

<tr><th>ID</th><th>Name</th><th>Email</th><th>Action</th></tr>

@foreach($students as $s)

<tr>

<td>{{ $s->id }}</td>

<td>{{ $s->name }}</td>

<td>{{ $s->email }}</td>

<td>

<a href="/students/edit/{{ $s->id }}">Edit</a>

<a href="/students/delete/{{ $s->id }}">Delete</a>

</td>
```

```
</tr>
@endforeach
</table>
```

(resources/views/students/create.blade.php)

```
<h2>Add Student</h2>
<form action="{{ route('students.store') }}" method="POST">
@csrf
<input type="text" name="name" placeholder="Name"><br>
<input type="email" name="email" placeholder="Email"><br>
<button type="submit">Save</button></form>
```

(resources/views/students/edit.blade.php)

```
<h2>Edit Student</h2>
<form action="{{ route('students.update', $student->id) }}" method="POST">
@csrf
<input type="text" name="name" value="{{ $student->name }}"><br>
<input type="email" name="email" value="{{ $student->email }}"><br>
<button type="submit">Update</button>
</form>
```

Mass No. 26

• Create a responsive card layout including an image, title, and description with hover effects. (5M)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Responsive Cards</title>
   <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-6">
   <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-6">
      <div class="bg-white rounded-xl shadow-md hover:shadow-2xl transition duration-300
overflow-hidden hover:-translate-y-1">
         <img src="https://via.placeholder.com/300" class="w-full h-48 object-cover">
         <div class="p-4">
```

```
            <h2 class="text-xl font-bold mb-2">Card Title</h2>
            <p class="text-gray-600">
                This is a responsive card with image, title, and description.
            </p>
        </div></div>
    <div class="bg-white rounded-xl shadow-md hover:shadow-2xl transition duration-300
overflow-hidden hover:-translate-y-1">
        <img src="https://via.placeholder.com/300" class="w-full h-48 object-cover">
        <div class="p-4">
            <h2 class="text-xl font-bold mb-2">Card Title</h2>
            <p class="text-gray-600">
                Tailwind CSS makes responsive design easy and fast.
            </p>
        </div>  </div>
    <div class="bg-white rounded-xl shadow-md hover:shadow-2xl transition duration-300
overflow-hidden hover:-translate-y-1">
        <img src="https://via.placeholder.com/300" class="w-full h-48 object-cover">
        <div class="p-4">
            <h2 class="text-xl font-bold mb-2">Card Title</h2>
            <p class="text-gray-600">
                Cards adjust automatically for mobile and desktop screens.
            </p>
        </div></div> </div>
</body></html>
```

• Create a form to upload multiple images, validate (image only), and store/display them. (10M)

Create Controller

```
php artisan make:controller ImageController
```

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class ImageController extends Controller
{
   public function index()
   {
      return view('images.upload');
   }

   public function store(Request $request)
   {
      $request->validate([
         'images.*' => 'required|image|mimes:jpg,jpeg,png,gif|max:2048'
```

```
        ]);
        $paths = [];
        if ($request->hasFile('images')) {
            foreach ($request->file('images') as $file) {
                $path = $file->store('images', 'public');
                $paths[] = $path;
            }
        }
        return back()->with('images', $paths);
    }
}
```

routes/web.php

```
use App\Http\Controllers\ImageController;

Route::get('/images/upload', [ImageController::class, 'index']);
Route::post('/images/upload', [ImageController::class, 'store'])->name('images.store');
```

Storage Link

```
php artisan storage:link
```

resources/views/images/upload.blade.php

```
<!DOCTYPE html>

<html>

<head>

    <title>Upload Images</title>

</head>

<body>

    <h2>Upload Multiple Images</h2>

    <form action="{{ route('images.store') }}" method="POST" enctype="multipart/form-data">

        @csrf

        <input type="file" name="images[]" multiple><br><br>

        @error('images.*')

            <p style="color:red;">{{ $message }}</p>

        @enderror

        <button type="submit">Upload</button>

    </form>

    @if(session('images'))

        <h3>Uploaded Images:</h3>

        @foreach(session('images') as $img)

            <img src="{{ asset('storage/' . $img) }}" width="150" style="margin:10px;">

        @endforeach

    @endif

</body></html>
```

• Create a Model and Controller for invoices table. Insert data and display all records. (10M)

Create Model + Migration

php artisan make:model Invoice -m

database/migrations/...create_invoices_table.php

```
public function up()
{
    Schema::create('invoices', function (Blueprint $table) {
        $table->id();
        $table->string('customer_name');
        $table->string('product');
        $table->integer('amount');
        $table->timestamps();
    });
}
```

php artisan migrate

app/Models/Invoice.php

```
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
class Invoice extends Model
{
    protected $fillable = ['customer_name', 'product', 'amount'];
}
```

Create Controller

php artisan make:controller InvoiceController

Controller Code

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Invoice;
class InvoiceController extends Controller
{
    public function index()
    {
        $invoices = Invoice::all();
        return view('invoices.index', compact('invoices'));
    }
    public function create()
    {
        return view('invoices.create');
    }
    public function store(Request $request)
    {
        $request->validate([
            'customer_name' => 'required',
            'product' => 'required',
            'amount' => 'required|numeric'
        ]);

        Invoice::create($request->all());
        return redirect('/invoices')->with('success', 'Invoice added');
    }
}
```

routes/web.php

```
use App\Http\Controllers\InvoiceController;

Route::get('/invoices', [InvoiceController::class, 'index']);
Route::get('/invoices/create', [InvoiceController::class, 'create']);
Route::post('/invoices', [InvoiceController::class, 'store'])->name('invoices.store');
```

resources/views/invoices/create.blade.php

```
<h2>Add Invoice</h2>

<form action="{{ route('invoices.store') }}" method="POST">

@csrf

<input type="text" name="customer_name" placeholder="Customer Name"><br>

<input type="text" name="product" placeholder="Product"><br>

<input type="text" name="amount" placeholder="Amount"><br>

<button type="submit">Save</button>

</form>
```

resources/views/invoices/ index.blade.php

```
<h2>All Invoices</h2>

<a href="/invoices/create">Add Invoice</a>

<table border="1">

<tr><th>ID</th><th>Customer</th><th>Product</th><th>Amount</th></tr>

@foreach($invoices as $inv)

<tr>

<td>{{ $inv->id }}</td>

<td>{{ $inv->customer_name }}</td>

<td>{{ $inv->product }}</td>

<td>{{ $inv->amount }}</td>

</tr>

@endforeach


</table>
```